Music Genre Classification from Stem Mashups

IIT Madras BS Degree
DL & GenAI Project (Jan 2026)

Name: Rohan Uday Puntambekar
Roll No: 21f3002298

GitHub Repository Link
Deployed Model Link (on HuggingFace)
Music Genre Classifier Streamlit App

Abstract

This report presents a music genre classification system developed for the Messy Mashup Kaggle competition, hosted as part of the IIT Madras BS Degree DL & GenAI course. The task requires classifying 10-second audio mashups into one of ten genres, where the test set is constructed by recombining stems from different songs of the same genre with added noise, creating a train-test distribution gap. The report explores six models across the machine learning and deep learning spectrum: a Decision Tree baseline, a CNN, a CRNN, EfficientNet-B0, MERT-v1-95M, and the Audio Spectrogram Transformer (AST). The central methodological contribution is a cross-song stem mixing augmentation strategy that replicates the test mashup construction process during training. Full fine-tuning of the AST on 400 augmented samples per genre achieved a Kaggle F1 score of 0.9314. The key finding is that augmentation design and training data diversity are more decisive than model architecture for this task.

1 Introduction

1.1 Problem Statement

The Messy Mashup competition [1] poses a 10-class music genre classification task across blues, classical, country, disco, hiphop, jazz, metal, pop, reggae, and rock genres. The training set consists of 1,000 songs (100 per genre), each decomposed into four stems: drums, vocals, bass, and other. The test set contains 3,020 mashups constructed by recombining stems from different songs of the same genre, applying tempo variations, adjusting instrument balances, and adding background noise from the ESC-50 dataset [2]. The evaluation metric is macro-averaged F1 score.

The central challenge is the distributional shift between training and test data. No single song contributes all four stems to any test mashup, which means a model trained naively on full songs will encounter combinations it has never seen. Bridging this gap requires one to apply various data augmentation and transfer learning approaches.

1.2 Project Objectives

The primary objective was to systematically compare a range of model families, from classical machine learning (ML) through hand-crafted features to self-supervised audio transformers, in order to understand which components of the pipeline contribute most to performance. A secondary objective was to gain practical experience with the HuggingFace Transformers library, PyTorch fine-tuning workflows, and experiment tracking via Weights & Biases. The final trained model is deployed as an interactive web application using Streamlit.

1.3 Report Structure

Section 2 describes the dataset and findings from exploratory analysis. Section 3 details the augmentation strategy used to bridge the train-test distribution gap. Section 4 presents each model architecture in depth. Section 5 summarises training configurations and hyperparameters. Section 6 reports evaluation results across all models. Section 7 provides an error analysis and Section 8 discusses the key findings and their implications. Section 9 describes the deployment of the interactive application and finally Section 10 concludes with broader insights and directions for future work.

2 Data and Exploratory Analysis

2.1 Dataset Descriptions

The dataset comprises 1,000 songs drawn from ten music genres (blues, classical, country, disco, hiphop, jazz, metal, pop, and rock), with 100 songs per genre. Each song is provided as four separated stems — drums, vocals, bass, and other — stored as WAV files at a native sample rate of 44,100 Hz with a duration of approximately 30 seconds per stem. The stems are organised under `data/messy_mashup/genres_stems/` with one subdirectory per song. The test set consists of 3,020 mashup files under `data/messy_mashup/mashups/`, each constructed by drawing individual stems from different songs of the same genre, applying tempo adjustments, and adding environmental noise sampled from the ESC-50 dataset [2], which contains 2,000 five-second clips across 50 acoustic categories. The test mashups are referenced by an integer ID in `test.csv`, with filenames following the pattern `songid:04d.wav`. No genre labels are provided for the test set; performance is evaluated via macro-averaged F1 score on the Kaggle leaderboard.

2.2 Exploratory Analysis

Exploratory analysis confirmed that the dataset contains no corrupted or zero-byte files. All 100 songs per genre are complete with all four stems present. Spectral analysis of the training stems revealed that blues and rock are one of the hardest genre pair to separate, with similar spectral centroids (1561 Hz and 1547 Hz respectively) and zero-crossing rates (0.07 and 0.06). Disco and metal seem comparatively easy to distinguish due to their higher spectral centroids (2928 Hz and 2776 Hz) and distinctive tempo profiles. The energy distributions of training stems and test mashups were found to be closely matched (train: -40.41 ± 5.81 dB; test: -41.29 ± 5.13 dB), confirming that energy normalisation alone would not resolve the distribution gap.

3 Augmentation Strategy

The key insight driving the augmentation design is that each test mashup is constructed by drawing stems from different songs of the same genre. Replicating this process during training would help address the distribution shift. For each training sample, each of the four stems is independently drawn from a randomly selected song of the target genre. A per-stem random gain sampled uniformly from $[0.5, 1.5]$ is applied before summation, simulating instrument balance variations. Background noise from ESC-50 is added at a randomly sampled SNR between 5 and 20 dB. Time-stretching was evaluated but removed because it caused mel-spectrogram time dimension instability and was prohibitively slow.

All models use 10-second audio clips at their respective sample rates. Features are precomputed and saved to disk before training to avoid repeating expensive audio processing each epoch.

4 Model Architectures

Six models were developed and evaluated across the project, ranging from a classical ML baseline to full transformer fine-tuning.

4.1 Decision Tree (Baseline)

A Decision Tree classifier (Milestone-2) [3] trained on hand-crafted spectral features extracted from the `other.wav` stem serves as the classical ML baseline. Four features are computed per file using `librosa`: tempo (beats per minute from beat tracking), spectral centroid (frequency-weighted mean of the power spectrum), zero-crossing rate (fraction of samples where the signal changes sign), and spectral rolloff (frequency below which 85% of spectral energy is concentrated). The tree uses a maximum depth of 5 to limit overfitting, trained on 50 songs per genre with a stratified 80/20 split. Despite its simplicity, the model establishes a useful lower bound and confirms that genre information is at least partially recoverable from single-stem spectral features.

4.2 Convolutional Neural Network

A four-block CNN (Milestone-3) processes log-mel spectrograms of the mixed audio, with each block following the pattern Conv2d → BatchNorm → ReLU → MaxPool2d. The channel progression is $1 \to 16 \to 32 \to 64 \to 128$, and each max pooling operation halves both the frequency and time dimensions. An `AdaptiveAvgPool2d` layer collapses the remaining spatial dimensions to a fixed-size vector regardless of input length, which is then passed to a Linear(128→10) classifier. Audio is mixed from individual song stems at 22050 Hz, converted to a log-mel spectrogram with 64 mel bins and a 5-second duration, and peak normalised before training.

4.3 CRNN (from scratch)

The primary from-scratch model is a Convolutional Recurrent Neural Network that treats genre classification as a sequential problem. The input is a log-mel spectrogram of shape $(1, 128, T)$, where T is the number of time frames.

Three convolutional blocks process the input, each consisting of Conv2d → BatchNorm → ReLU → MaxPool2d(2×2) → Dropout2d(0.3). The channel widths increase as $1 \to 32 \to 64 \to 128$. After three max pooling operations, the mel frequency dimension reduces from 128 to $128/2^3 = 16$. The output is permuted from shape $(B, 128, T', 16)$ to $(B, T', 128 \times 16)$, giving a time sequence of 2048-dimensional feature vectors.

This sequence is fed into a two-layer bidirectional LSTM with hidden size 128. Being bidirectional, the effective hidden dimension per timestep is 256. Global max pooling over the time dimension collapses the sequence to a single 256-dimensional vector, which is passed to a Linear(256→10) classifier. Audio is processed at 22050 Hz with $N_{\text{FFT}} = 2048$, hop length 256, and 128 mel bins over a 10-second duration. The spectrogram is converted to dB scale via `power_to_db` and then min-max normalised to $[0, 1]$. Training uses AdamW ($\text{lr} = 3 \times 10^{-4}$, weight decay 10^{-4}) with label smoothing cross-entropy (smoothing factor 0.1) and cosine annealing over 40 epochs.

4.4 EfficientNet-B0

EfficientNet-B0 [4], pretrained on ImageNet, was adapted for audio classification as a third model. The mel-spectrogram is computed with the same parameters as the CRNN (SR=22050, 128 mel bins, hop length 256, 10 seconds). Since EfficientNet expects a 3-channel image input, the single-channel spectrogram is replicated across all three channels and resized to 224×224 using bilinear interpolation. ImageNet mean and standard deviation normalisation is applied before feeding to the network.

The original EfficientNet-B0 classifier head (Linear(1280→1000)) is replaced with Dropout(0.2) followed by Linear(1280→10). Fine-tuning was done in two phases: Phase 1 trains the new head only with the backbone frozen, and Phase 2 additionally unfreezes the final MBConv block (`features[-1]`) alongside the head, using differential learning rates. This model was trained and evaluated for completeness but not submitted to Kaggle due to its lower performance relative to the AST models.

4.5 Audio Spectrogram Transformer (AST)

The pretrained AST model [5], specifically the `MIT/ast-finetuned-audioset-10-10-0.4593` checkpoint, was fine-tuned on this task. The AST applies the Vision Transformer architecture [6] to audio by treating a log-mel spectrogram as an image. Audio is resampled to 16000 Hz and the HuggingFace `AutoFeatureExtractor`

produces a spectrogram of shape 1024×128 (time frames $\times$ mel bins). This is divided into 16×16 non-overlapping patches, yielding $\lfloor 1024/16 \rfloor \times \lfloor 128/16 \rfloor = 64 \times 8 = 512$ patches. Each patch is flattened and projected to a 768-dimensional embedding via a learned linear projection. Two additional learnable tokens (distillation and classification) are prepended, and sinusoidal positional embeddings are added. The resulting 514-token sequence is processed by 12 transformer encoder blocks, each with 12 attention heads, a feedforward hidden size of 3072, and GELU activations. The classification token's output is passed to the classification head. The original 527-class head (pre-trained on AudioSet) is replaced with Linear(768→10).

Four fine-tuning configurations were explored progressively. Phase 1 trains only the new head (9,226 trainable parameters) for 5 epochs. Phase 2 unfreezes the two deepest encoder blocks and applies differential learning rates for 10 epochs. Phase 3 continues from the Phase 2 checkpoint, unfreezing three encoder blocks at further reduced rates. Finally, full fine-tuning of all 87 million parameters was performed on Google Colab (T4 GPU) with a uniform learning rate of 10^{-5} and cosine annealing over 10 epochs. Gradient norms are clipped to 1.0 throughout. To prevent overfitting and increase the diversity in the training dataset, the full finetuned model was trained on 400 augmented samples per genre instead of the 200 augmented samples (per genre) used for rest of the models.

4.6 MERT

MERT-v1-95M [7], a music-specific self-supervised transformer pretrained on large-scale music corpora using acoustic teacher signals, was fine-tuned on Colab. Unlike AST, MERT operates directly on raw waveforms rather than spectrograms. Audio is resampled to 24000 Hz and per-sample mean-variance normalisation is applied before feeding into the model. The backbone consists of a convolutional feature encoder followed by 12 transformer encoder blocks with hidden size 768. The last hidden state, of shape $(B, T, 768)$, is mean-pooled over the time dimension T to produce a fixed-length 768-dimensional representation, which is passed to a Linear(768→10) classification head.

Fine-tuning follows the same two-phase strategy as AST: Phase 1 trains the head only for 5 epochs (lr = 10^{-3}), and Phase 2 unfreezes the two deepest encoder blocks for 10 more epochs with differential learning rates (head: 10^{-4}, trunk: 10^{-5}). Training used 350 augmented samples per genre at batch size 8, constrained by GPU memory.

5 Training Details and Hyperparameters

Table 1: Training configuration per model.

Model	Optimiser	Key Config
Decision Tree	–	max_depth=5
CNN	AdamW	lr=1e-3, 30 epochs
CRNN	AdamW	lr=3e-4, 40 epochs
AST (phase-wise)	AdamW	lr_head=1e-4, lr_trunk=1e-5
AST (full)	AdamW	lr=1e-5, 10 epochs
MERT	AdamW	lr_head=1e-4, lr_trunk=1e-5

All deep learning experiments are logged on Weights & Biases [8] under the project `21f3002298-dl-genai-project`. The best validation checkpoint is saved at each epoch. Test-time augmentation (TTA) is applied during inference: five evenly spaced 10-second crops are extracted from each test mashup, softmax probabilities are averaged, and the argmax is taken as the final prediction.

6 Results

Table 2 summarises validation and Kaggle F1 scores across all models and training configurations. The full fine-tune of AST on 400 augmented samples per genre achieved the best Kaggle score of 0.9314.

Table 2: Model performance. Val F1 is on the held-out augmented validation set. Kaggle F1 is the public leaderboard score.

Model	Val F1	Kaggle F1
Decision Tree	0.1523	–
CNN (milestone)	0.7271	0.4355
CRNN (milestone)	0.5462	0.1834
CRNN (src)	0.6661	0.5340
EfficientNet-B0	0.6119	–
MERT phase 2	0.7531	–
AST frozen (milestone)	0.6588	0.6431
AST phase 1	0.7663	0.7624
AST phase 2	0.8026	0.7890
AST phase 3	0.8042	–
Ensemble (AST P2 + CRNN)	–	0.8035
Ensemble (AST P3 + MERT)	–	0.8147
AST full fine-tune (200/genre)	0.8618	0.8567
AST full fine-tune (400/genre)	**0.9763**	**0.9314**

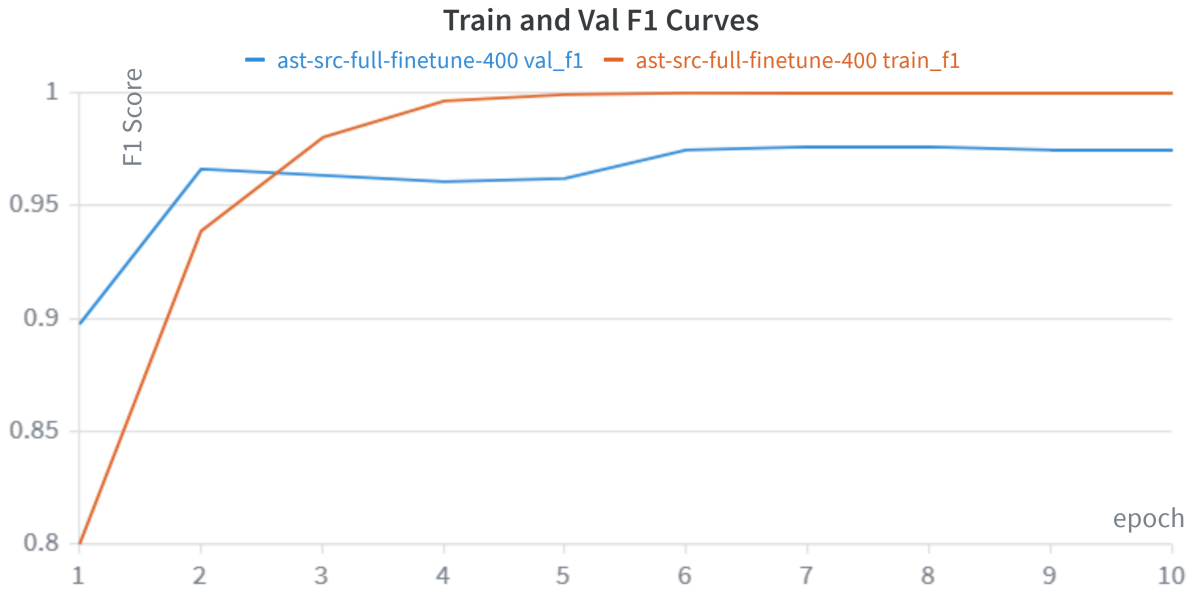


Figure 1: Training and validation F1 curves for the best AST full fine-tune run (400 samples/genre). Val F1 peaks at 0.9763 at epoch 7 before plateauing.

7 Error Analysis

The progression of models across the project reveals patterns in which genres are difficult to classify and how different architectures handle them.

Decision Tree. The classification report shows that four genres (classical, hiphop, pop, rock) received zero true positives entirely. The model collapsed predictions towards disco and country, which had higher representation in the feature space. This confirms that single-stem spectral features extracted from `other.wav` carry insufficient discriminative information for most genres, and that the hand-crafted feature set cannot capture the full range of genre variation.

CNN. Moving to mel-spectrogram features improved performance substantially. The per-genre classification report shows classical and country achieving a precision of 1.00 and metal performing well, while rock remained the weakest class with precision 0.50 and recall 0.35. Country showed a particularly sharp precision-recall imbalance: precision of 1.00 but recall of only 0.47, indicating the model was conservative about predicting country and missed many true positives. This pattern suggests the CNN was learning narrow, high-confidence features for some genres while failing to generalise for others.

CRNN. The confusion matrix (Figure 2) highlights where errors concentrate. Classical achieves near-perfect classification (40/40). Pop is also strong (48/51). Rock is the most weakly predicted class: only 14 of 43 samples are correctly classified, with 9 misclassified as disco and 7 as blues. Jazz is the second most confused class (25/44 correct), scattering into blues (3), classical (6), country (3), and disco (3). Country struggles too (18/40), confused frequently with blues and disco. This suggests that rhythmically and spectrally similar genres are the main failure modes for sequence-based models on this task.

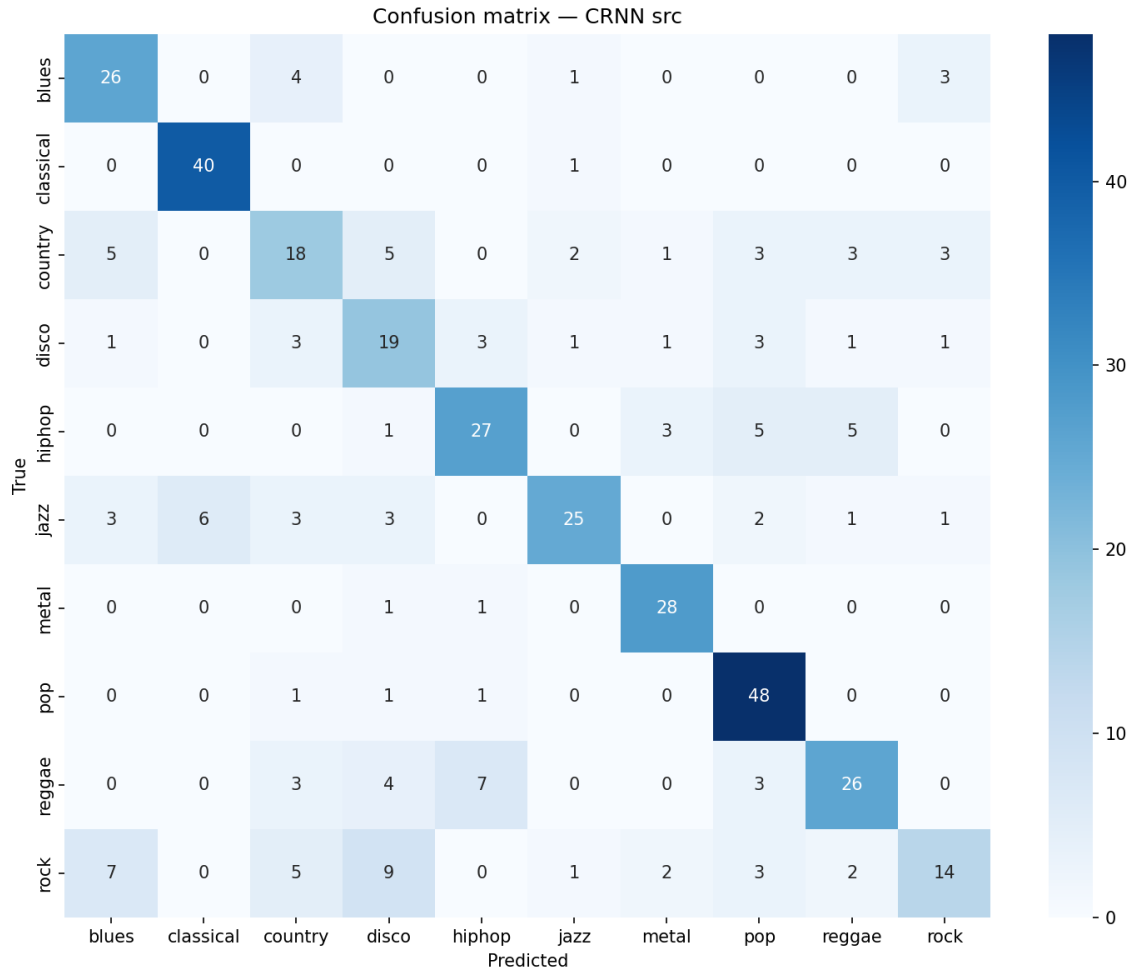


Figure 2: Confusion matrix for the CRNN src model (val F1 = 0.6661).

EfficientNet-B0. The EfficientNet confusion matrix (Figure 3) shows a similar error structure to the CRNN but with some differences. Metal improves substantially (36/40) compared to the CRNN. However, blues is now the weakest class (18/40), with 6 samples confused with jazz — the reverse of the CRNN's jazz-to-blues errors. Rock remains weak (12/40), confused with blues (6). The jazz-blues and rock-country axes account for the majority of off-diagonal mass in both models, suggesting these misclassifications are a structural property of the task rather than a specific model failure.

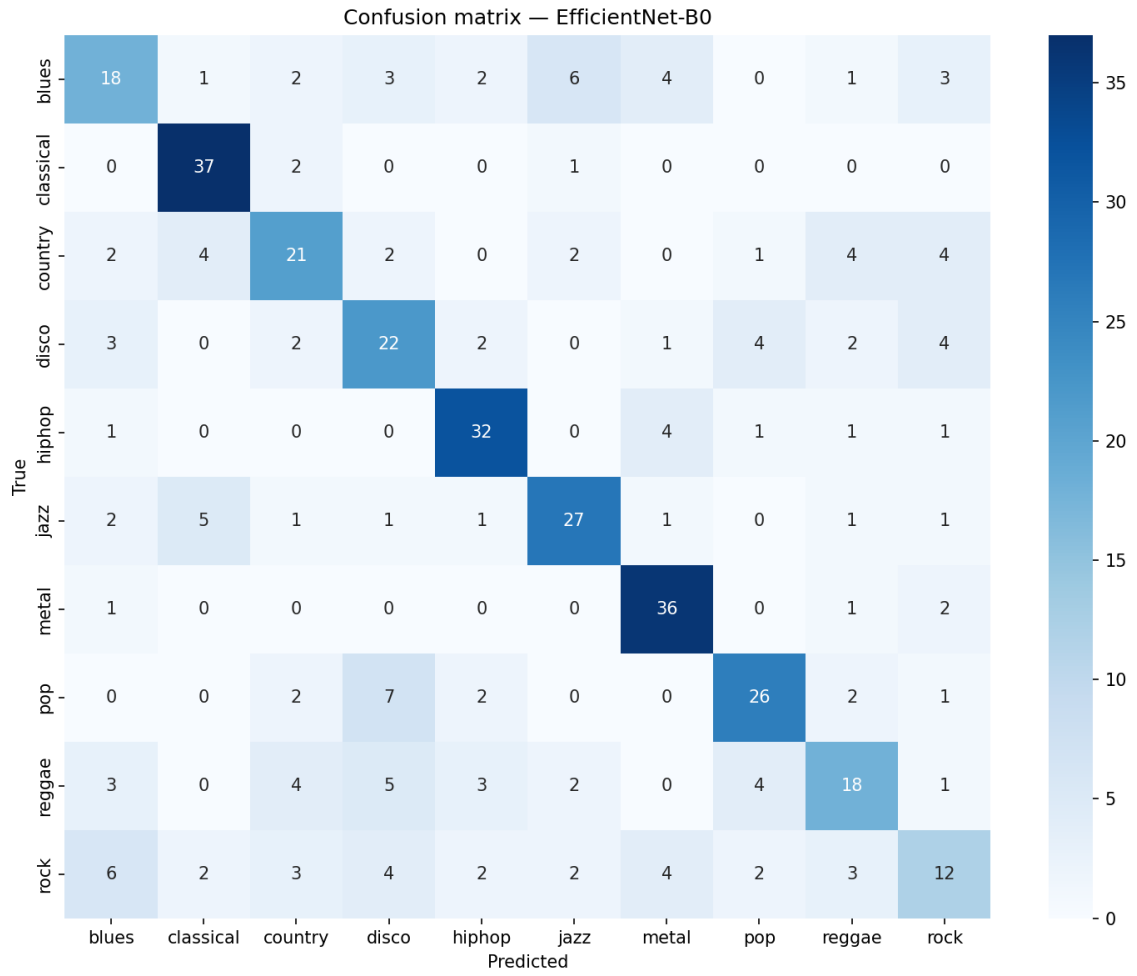


Figure 3: Confusion matrix for EfficientNet-B0 (val F1 = 0.6119).

AST fine-tuning progression. Moving to the AST architecture and progressively unfreezing more transformer blocks produced consistent Kaggle improvements (phase 1: 0.7624, phase 2: 0.7890, ensemble with CRNN: 0.8035, ensemble with MERT: 0.8147). Each step of unfreezing allowed the model to adapt lower-level audio representations more specifically to genre-discriminative features, which the frozen-head configuration could not achieve.

Full fine-tuning. Increasing augmented training samples from 200 to 400 per genre while fully fine-tuning the AST produced the largest single jump in the project (Kaggle F1: 0.8567 to 0.9314). The 4.5-point gap between validation F1 (0.9763) and Kaggle F1 (0.9314) highlights that the test set introduces additional variation not fully captured by augmentation, but the gap is modest and consistent with the level of generalisation seen across the model progression.

8 Discussion

The most consequential design decision was the cross-song stem mixing augmentation. Moving from single-song feature extraction (milestone pipeline) to independently sampling each stem from a different song of the same genre replicated the test mashup construction process to a certain extent. This change, combined with the full fine-tuning of AST, drove the largest performance jumps in the project.

Increasing the number of training samples from 200 to 400 per genre produced a further 7.5 point absolute improvement in Kaggle F1 (0.8567 to 0.9314). This is notable because the underlying architecture and

learning rate did not change between the two runs. It suggests the model was previously limited by training data diversity rather than model capacity.

The MERT model, despite being music-specific in its pretraining, achieved a lower Kaggle F1 than AST. A likely reason is that MERT's pretraining used a 24000 Hz sample rate, while most of the genre-discriminative information in these stems is concentrated at frequencies well below 12000 Hz. The AST's AudioSet pretraining, which covers a broader range of acoustic events at 16000 Hz, may provide more transferable representations for this task.

9 Deployment

The best model is deployed as an interactive web application using Streamlit, accessible at https://music-genre-classifier-app-nxjyfsazbfkpqqzwpujfyw.streamlit.app. Users can upload a WAV or MP3 file and receive the top 5 predicted genres with confidence scores visualised as a horizontal bar chart. Predictions are produced using the same 5-crop TTA pipeline used during Kaggle inference.

The model weights are hosted on HuggingFace Hub at https://huggingface.co/Rohhnn/music-genre-classifier and are downloaded at application startup via the `huggingface_hub` library. This avoids storing large binary files in the GitHub repository. The deployment code is maintained in a separate public repository at https://github.com/Rohhnn/music-genre-classifier-app, while the full training pipeline resides in the private project repository at https://github.com/Rohhnn/dl-genai-project-25-t3.

10 Conclusion

Several broader insights emerge from this project. First, the quality of the augmentation strategy matters more than model capacity at the lower end of the model spectrum. The CRNN and EfficientNet both plateau around 0.61–0.67 val F1 regardless of architectural refinements, while the same augmentation pipeline applied to a stronger pretrained model yields dramatically better results. Second, pretraining domain matters but is not decisive. MERT is pretrained specifically on music, yet AST (pretrained on general audio) outperforms it here, likely because AudioSet's diversity provides broader acoustic coverage that transfers well to stem-based genre signals. Third, the most significant insight for improving Kaggle scores throughout the project was increasing training sample diversity, not model complexity.

Future work could explore several directions. The jazz-blues and rock-country confusion axes are consistent across models, suggesting these genre pairs are ambiguous in the stem-mashup format. Targeted augmentation that increases within-pair diversity, or auxiliary losses that explicitly penalise these confusions, could help. On the data side, applying more aggressive time-stretch and pitch-shift augmentation (which was removed due to computational cost) may close the remaining gap between validation and test performance. Finally, a student-teacher distillation setup, where the full fine-tuned AST serves as the teacher for a smaller student model, would make deployment more practical without sacrificing much accuracy.

References

[1] IIT Madras DL & GenAI Project Team. Jan 2026 dlgenai project - messy mashup. Kaggle Competition, 2026. URL https://www.kaggle.com/competitions/jan-2026-dl-gen-ai-project.

[2] Karol J Piczak. Esc: Dataset for environmental sound classification. In *Proceedings of the 23rd ACM international conference on Multimedia*, pages 1015–1018, 2015.

[3] Leo Breiman, Jerome Friedman, Richard A Olshen, and Charles J Stone. *Classification and regression trees*. Chapman and Hall/CRC, 2017.

[4] Mingxing Tan and Quoc Le. Efficientnet: Rethinking model scaling for convolutional neural networks. In *International conference on machine learning*, pages 6105–6114. PMLR, 2019.

[5] Yuan Gong, Yu-An Chung, and James Glass. Ast: Audio spectrogram transformer. *arXiv preprint arXiv:2104.01778*, 2021.

[6] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.

[7] Yizhi Li, Ruibin Yuan, Ge Zhang, Yinghao Ma, Xingran Chen, Hanzhi Yin, Chenghao Xiao, Chenghua Lin, Anton Ragni, Emmanouil Benetos, et al. Mert: Acoustic music understanding model with large-scale self-supervised training. *arXiv preprint arXiv:2306.00107*, 2023.

[8] Lukas Biewald et al. Experiment tracking with weights and biases. 2020.